Inferential Statistics — R Practical

Part 3 · 307312 Applied Statistics

How we will use this notebook

- **Purpose:** build a record of calculations, interpretations and independent work as we progress through Part 3.
- **Available now:** probability distributions, including seven worked R examples and an independent activity. Estimation, testing and regression will be added as those sections are developed.
- **Setup:** install R, RStudio and Quarto. Rendering requires knitr and rmarkdown; these calculations use base R and need no extra statistics package or CSV.
- **Using the course project:** open `Applied-Statistics.Rproj`, then this notebook under `practicals/`. Keep the project folder as your working directory.
- **Using the individual download:** save the Quarto file in a working folder and open it in RStudio. This probability section is self-contained.
- **Each time:** read the explanation, predict the result, run the cell, then write a sentence interpreting the output. Keep your written responses in your copy of the notebook.
- **Data:** die outcomes are enumerated; distribution inputs are stipulated models. Simulated scores are generated values, not observed student records.

Probability distributions

The examples follow Section 3.1 in teaching order. Functions and arguments are explained before their code; refer to the side contents panel to move between activities.

1. Equally likely outcomes: logical tests and proportions

A **discrete uniform distribution** assigns the same probability to each of a finite set of possible values. For a fair six-sided die, each outcome has probability $1/6$.

Event	Included outcomes	Probability
Roll a 3	3	$1/6 = 0.1667$
Roll an even number	2, 4, 6	$3/6 = 0.5$
Roll from 2 to 5, inclusive	2, 3, 4, 5	$4/6 = 0.6667$

For equally likely integers $a, a+1, \ldots, b$:

$$p(x) = \frac{1}{b-a+1}.$$

How to read the R code

- `outcomes <- 1:6` stores the six possible values (1 through 6) in `outcomes`; `<-` means "assign."
- `outcomes == 3` checks each value and returns `FALSE FALSE TRUE FALSE FALSE FALSE`. `==` tests equality.
- `mean()` treats `TRUE` as 1 and `FALSE` as 0. Here it gives $(0+0+1+0+0+0)/6 = 1/6$: the **proportion satisfying the condition**, not the mean die score.
- `%%` gives the remainder after division: `outcomes %% 2 == 0` selects even values. `&` requires both conditions to hold; `>=` and `<=` include the endpoints.

- `c()` combines results into one vector. Labels such as `roll_3 =` name the results in the output.

These proportions are probabilities because **all six outcomes are equally likely**.

```
outcomes <- 1:6
c(roll_3 = mean(outcomes == 3),
  even = mean(outcomes %% 2 == 0),
  from_2_to_5 = mean(outcomes >= 2 & outcomes <= 5))
```

```
    roll_3       even from_2_to_5
0.1666667  0.5000000  0.6666667
```

Equivalent calculation: `sum(outcomes == 3) / length(outcomes)` counts the matching values and divides by the total number of values. `sum()` adds the 1s and 0s; `length()` counts all entries.

Fairness is stipulated here; it is not established by one roll.

2. Binomial probabilities: exact counts and cumulative totals

Context: X counts defective cars in a batch of 80. Assume independent defect outcomes and a common defect probability of 0.02 for each car.

An **exact-count probability** concerns one value, such as $X = 2$. A **cumulative probability** adds the probabilities of all counts up to a cutoff, such as $P(X \leq 2)$. For the same car model, $n = 80$ and $p = 0.02$.

$$P(X = x) = \binom{n}{x} p^x (1-p)^{n-x}.$$

Question	Event	Probability
Exactly two defective cars	$X = 2$	0.26145
Fewer than two	$X = 0$ or $X = 1$	0.52297
More than two	$X \geq 3$	0.21558
No defective cars	$X = 0$	0.19865

Interpretation: exactly two defects has a probability of about **26.14%** for a batch under this model. “More than two” starts at three, so it excludes two.

R functions: what they calculate

- `dbinom(x, size, prob)` returns $P(X = x)$: the probability of **exactly** `x` successes. `binom` identifies the binomial distribution; R uses the `d` prefix for mass/density functions. Here the result is a probability mass.
- `pbinom(q, size, prob)` returns $P(X \leq q)$ by default: the **cumulative probability up to and including** cutoff `q`. The `p` prefix identifies a cumulative probability function.
- `size=80` is the number of trials; `prob=0.02` is the success probability **on each trial**, not the requested batch probability.
- `lower.tail=FALSE` changes the cumulative calculation to $P(X > q)$. Thus “fewer than two” uses cutoff **1**, while “more than two” uses cutoff **2** and the upper tail.
- `c()` combines the four answers; the names before `=` label the output.

```
c(exactly_2 = dbinom(x=2, size=80, prob=0.02),
  fewer_than_2 = pbinom(q=1, size=80, prob=0.02),
  more_than_2 = pbinom(q=2, size=80, prob=0.02,
                       lower.tail=FALSE),
  no_defects = dbinom(x=0, size=80, prob=0.02))
```

```
exactly_2 fewer_than_2  more_than_2   no_defects
0.2614454    0.5229735    0.2155811    0.1986489
```

Excel: `BINOM.DIST(2,80,0.02,FALSE)` gives exactly two. Use `TRUE` for the cumulative probability $P(X \le x)$.

3. Poisson probabilities: arrivals in a minute

Context: X counts bank arrivals in one minute. Assume a constant rate of three per minute and independent event occurrences.

The **complement rule** gives an event's probability by subtracting the probability of everything outside it from 1. For the bank model with $\lambda = 3$, "more than two" is the complement of "two or fewer."

$$P(X = x) = e^{-\lambda}\frac{\lambda^x}{x!}.$$

- Exactly two: $P(X = 2) = 0.22404$, approximately **22.40%**.
- More than two: $P(X > 2) = 1 - P(X \le 2) = 0.57681$, approximately **57.68%**.

R functions: exact count versus cumulative probability

- `dpois(x, lambda)` returns $P(X = x)$: exactly `x` events. `pois` identifies the Poisson distribution; `d` gives its probability mass.
- `ppois(q, lambda)` returns $P(X \le q)$ by default. `p` means cumulative probability; `lower.tail=FALSE` instead returns $P(X > q)$.
- `lambda=3` is the expected number of arrivals **in one minute**. The first answer uses count `x=2`; the second uses cutoff `q=2`. `c()` collects and labels the answers.

```
c(exactly_2 = dpois(x=2, lambda=3),
  more_than_2 = ppois(q=2, lambda=3, lower.tail=FALSE))
```

```
exactly_2 more_than_2
0.2240418   0.5768099
```

Excel: `POISSON.DIST(2,3,FALSE)` and `1-POISSON.DIST(2,3,TRUE)`.

Interpretation: under this model, about 57.68% of one-minute intervals have more than two arrivals in the long run. "More than two" excludes two; "two or more" includes it.

4. Uniform probabilities: delivery times and diameters

Under a uniform model, the proportion above a threshold equals the length of the interval above that threshold divided by the full interval length. We now apply that rule to a measurement in millimetres.

Assume component diameter D is uniform from 9.8 to 10.2 mm.

Question: what proportion exceeds 10.1 mm?

$$P(D > 10.1) = \frac{10.2 - 10.1}{10.2 - 9.8} = 0.25.$$

Quantity	Calculation	Result
Mean diameter	$(9.8 + 10.2)/2$	10.0 mm
Diameter variance	$(10.2 - 9.8)^2/12$	0.013333 mm^2
Above 10.1 mm	Interval length 0.1 divided by 0.4	25%

R function: cumulative uniform probability

- `punif(q, min, max)` returns the probability at or below cutoff `q` under a continuous uniform model. `p` means cumulative probability; `unif` identifies the uniform distribution.
- `min` and `max` are the model's lower and upper limits, **not values R estimates from data**. For arrival time, `q=9`, `min=8` and `max=12` are hours.
- `lower.tail=FALSE` requests the probability **above** `q`. For diameters, all three values are in millimetres.
- For a continuous model, including the cutoff does not change the probability. `c()` combines two separate answers, both 0.25; the labels identify their contexts.

```
c(arrival_before_9 = punif(q=9, min=8, max=12),
  diameter_above_10_1 = punif(q=10.1, min=9.8, max=10.2,
                              lower.tail=FALSE))
```

```
  arrival_before_9 diameter_above_10_1
              0.25                0.25
```

- Uniformity is stipulated, not established by the fact that a machine made the components.
- "Above 10.1 mm" is not automatically a defect definition; a specification must be supplied separately.

5. Generate a simulated dataset: the four R function families

R provides **four related functions** for many distributions. The prefix tells us the task; the ending identifies the distribution. For the normal model, the ending is `norm`.

Function	What it returns	Main input
`dnorm(x, mean, sd)`	Density height at a value; **not** the probability of that exact value.	Value `x`
`pnorm(q, mean, sd)`	Probability at or below a cutoff, $P(X \le q)$.	Cutoff `q`
`qnorm(p, mean, sd)`	The cutoff with cumulative probability `p` below it; a **quantile**.	Probability `p`, between 0 and 1
`rnorm(n, mean, sd)`	`n` simulated observations from the specified normal model.	Number of observations `n`

Create a small simulated dataset

- `set.seed(312)` makes the random sequence reproducible when this code is rerun in the same R setup. The seed is a chosen starting value, not a sample size.
- `rnorm(n=12, mean=70, sd=6)` generates 12 scores. `data.frame(score=...)` puts them in a table with one column named `score`; `<-` stores that table as `simulated_scores`. `head()` displays its first six rows.

```
set.seed(312)
simulated_scores <- data.frame(score=rnorm(n=12, mean=70, sd=6))
head(simulated_scores)
```

```
     score
1 74.87287
2 79.04831
3 75.48955
4 65.52904
5 69.41006
6 63.77732
```

Read the output: each row is one simulated observation. These are generated values, not collected student records. The model mean is 70 and SD is 6; a sample of 12 will generally have a different mean and SD.

Predict before trying: `qnorm(p=0.90, mean=70, sd=6)` returns the score below which the model places 90% of values. It reverses the question answered by `pnorm()`.

The same naming pattern applies to `dbinom` / `pbinom` / `qbinom` / `rbinom`, `dpois` / `ppois` / `qpois` / `rpois` and `dunif` / `punif` / `qunif` / `runif`. For discrete models, `d` returns probability mass.

6. Standardise scores: compare relative positions

Comparing **z-scores** shows how far each result lies above or below its own group's mean. This lets us compare relative positions when the groups have different means or spreads.

Khalid scores 90 in both Python and Database.

Course	Mean	SD	Score	z-score
Python	70	10	90	$(90 - 70)/10 = 2$
Database	80	10	90	$(90 - 80)/10 = 1$

- His score is **two SDs above the Python mean**, compared with **one SD above the Database mean**. His relative position is higher in Python.
- The same raw score need not indicate the same relative position.
- This comparison does not show that the examinations measured identical knowledge or had equal difficulty.

Reading the R calculation: `(score - mean) / SD` implements the z-score formula; parentheses make the subtraction happen before division. `c()` puts the two calculated values into one vector, with `Python` and `Database` as output labels. It does not fit a model or calculate probabilities.

```r
c(Python = (90-70)/10, Database = (90-80)/10)
```

```
  Python Database
       2        1
```

Interpret in one sentence: what does a z-score of 2 tell us here?

7. Normal probabilities: thresholds and percentages

Excel and R calculate normal probabilities directly from a threshold, mean and SD. Both normally return the area below the threshold; to answer "above 75," request the upper tail or subtract the lower-tail probability from 1.

The question remains **score above 75**, with mean 70 and SD 6.

Excel

`=1-NORM.DIST(75,70,6,TRUE)`

R: what the function and arguments mean

- `pnorm(q, mean, sd)` returns the probability at or below cutoff `q` under a normal model. `p` means cumulative probability; `norm` identifies the normal distribution.
- `q=75` is the score threshold, `mean=70` sets the model's centre and `sd=6` sets its standard deviation, **not its variance**.
- `lower.tail=FALSE` requests $P(X > 75)$ instead of $P(X \leq 75)$. `<-` stores the result in `p`; `c()` displays it alongside `100*p`, its percentage.

```
p <- pnorm(q=75, mean=70, sd=6, lower.tail=FALSE)
c(probability=p,percentage=100*p)
```

```
probability  percentage
  0.2023284  20.2328381
```

Interpretation: the model assigns a probability of approximately **0.20233**, or **20.233%**, to an exam score above 75.

- `lower.tail=FALSE` asks for the area above the threshold.
- A probability of 0.20233 is not 0.20233%.
- Table rounding explains 20.33% versus the unrounded software result of 20.23%.
- For a normal interval, subtract two cumulative probabilities: for example, $P(64 < X < 76) = \Phi(1) - \Phi(-1) \approx 0.68269$.

8. Independent probability activity

Apply the same four steps to each question: **define the event, state the assumptions, calculate the probability and interpret the result**. Use the models and inputs given below.

1. **Car defects:** among 80 cars with independent defect probability 0.02, find the probability of **at least two** defective cars. How does it differ from "more than two"?
2. **Bank arrivals:** under the Poisson model with mean three per minute, find the probability of **no arrivals** in one minute.
3. **Deliveries:** under $U(8, 12)$, find the probability of an arrival between **10 and 11 a.m.**
4. **Exam scores:** under the normal model with mean 70 and SD 6, find the probability of a score **between 64 and 76**.

For each question:

- Define the random variable and event before opening R.
- State one model assumption that would matter in practice.
- Calculate using a suitable function.
- Write a sentence giving the probability and its context.

Explain: why does a correct software calculation not establish that the chosen model fits real observations?

Write your response below each question: define the event, give your R command, report the result and state the main assumption.

9. Your own simulated data

- Copy the simulated-score cell and change `n=12` to `n=100`. Keep `mean=70`, `sd=6` and the seed fixed.
- Record how many rows your table contains and explain what one row represents.

- `mean(simulated_scores$score)` calculates the arithmetic average of the score column; `$score` selects that column. Here `mean()` averages numerical values, rather than TRUE/FALSE conditions.
- `sd(simulated_scores$score)` calculates the sample standard deviation of those scores. Explain why the results need not equal the model inputs 70 and 6 exactly.
- Use `qnorm(p=0.90, mean=70, sd=6)` to obtain the model's 90th-percentile cutoff. Check it by supplying that cutoff as `q` to `pnorm(q=..., mean=70, sd=6)`; replace `...` with the calculated value.
- Explain why simulated scores cannot establish the actual distribution of scores in our class.

Estimation — to be developed

- We will add sample-based estimates and confidence-interval activities when this teaching section is ready.

Hypothesis testing and power — to be developed

- We will add tests, interpretation and the relationship between sample size, significance level and power when this teaching section is ready.

Regression — to be developed

- We will add model fitting, diagnostics, interpretation and prediction when this teaching section is ready.

Sources

- Companion to the revised probability-distributions section, using the original probability examples and Berenson, Levine and Krehbiel, *Basic Business Statistics: Concepts and Applications*, ninth edition, selected §§5.1, 5.3, 5.5 and 6.1–6.3.
- R explanations and simulation activities prepared for this course. No additional observed dataset is claimed for this probability section.